If already registered, click to check your payment status

Course outline

About NPTEL ()

How does an NPTEL online course work? ()

Week 1 ()

Week 2 ()

Week 3 ()

week 4 ()

Week 5 ()

Week 6 ()

Substitution Cipher -The science of secrecy (unit?unit=124&lesson=125)

Substitution Cipher -The

Thank you for taking the Week 6 : Assignment 6.

Week 6 : Assignment 6

Your last recorded submission was on 2025-09-03, 13:59 IST **Due date: 2025-09-03, 23:59 IST.**

Prof. Andrew is a computer science teacher at a high school. With the annual Parent-Teacher Meeting approaching, he wants to organize student data neatly using Python. He plans to keep track of students' marks, personal details, and house assignments.

Being a Python pro, Prof. Andrew considers two ways to represent this data effectively.

First, a Dictionary of Lists Prof. Andrew thinks to himself: *"I want to quickly check a student's marks just by typing their name."* So, he writes the following code:

```
marks = {
    "Aanya": [85, 90, 88],
    "Ravi": [78, 82, 80],
    "Zoya": [92, 95, 89]
}
```

Each key is a student's name, and the value is a list of their marks in three subjects: Math, Science, and English.

If he wants to know Ravi's Science mark (the second subject), he simply writes:

```
print(marks["Ravi"][1])
```

The output is 82.

Assessment submitted.
X

science of secrecy 01 (unit?unit=124&lesson=126)
Substitution Cipher -The science of secrecy 02 (unit?unit=124&lesson=127)
Substitution Cipher -The science of secrecy 03 (unit?unit=124&lesson=128)
Tic Tac Toe - Down the memory Lane (unit?unit=124&lesson=129)
Tic Tac Toe - Down the memory Lane 01 (unit?unit=124&lesson=130)
Tic Tac Toe - Down the memory Lane 02 (unit?unit=124&lesson=131)
Tic Tac Toe - Down the memory Lane 03 (unit?unit=124&lesson=132)
Tic Tac Toe - Down the memory Lane 04 (unit?unit=124&lesson=133)
Tic Tac Toe - Down the memory Lane

This structure works well when he needs to look up a **specific student** and view all their marks.

Next, a List of Dictionaries

Later, the school admin asks Prof. Andrew for a list of students along with their roll numbers and house names.

He now decides to create a different structure — one where **each student is a complete record.**

```
students = [
    {"name": "Aanya", "roll": 1, "house": "Red"},
    {"name": "Ravi", "roll": 2, "house": "Blue"},
    {"name": "Zoya", "roll": 3, "house": "Green"}
]
```

Now he can loop through each student's information:

```
for student in students:
    print(student["name"], "belongs to", student["house"], "house.")
```

This setup is ideal when working with multiple entries and needing to **process each student's data one by one** — similar to going through rows in a spreadsheet.

1) What data structure did Prof. Andrew use to store student marks where the key was the student's name and the value was a list of marks? ***1 point***

- ○ List of Lists
- ○ List of Dictionaries
- ◉ Dictionary of Lists
- ○ Dictionary of Dictionaries

2) What will be the output of the following code? ***1 point***

```
marks = {
    "Aanya": [85, 90, 88],
    "Ravi": [78, 82, 80],
    "Zoya": [92, 95, 89]
}
print(marks["Zoya"][0])
```

- ○ 85
- ◉ 92
- ○ 89

Assessment submitted.
X

05 (unit? unit=124&lesson=134)
Recursion (unit? unit=124&lesson=135)
Recursion 01 (unit? unit=124&lesson=136)
Recursion 02 (unit? unit=124&lesson=137)
Recursion 03 (unit? unit=124&lesson=138)
Recursion 04 (unit? unit=124&lesson=139)
Recursion 05 (unit? unit=124&lesson=140)
Recursion 06 (unit? unit=124&lesson=141)
Week 6 Feedback Form: The Joy of Computing using Python (unit? unit=124&lesson=142)
Quiz: Week 6 : Assignment 6 (assessment? name=580)
Week 6 Programming Assignment : 1 (/noc25_cs103/progassignment?name=581)

Error

3) Fill in the blank to print Ravi's English marks from the marks dictionary: ***1 point***

```
print(marks["Ravi"][___])
```

- (•) 2
- ( ) 0
- ( ) 1
- ( ) 3

4) Which of the following best describes when to use a list of dictionaries? ***1 point***

- ( ) When you want to access one student's data quickly
- ( ) When the values are numbers only
- (•) When you want to loop through each student's complete record
- ( ) When you are storing a single subject's marks

5) What will be the output of the following code? ***1 point***

```
students = [
    {"name": "Aanya", "roll": 1, "house": "Red"},
    {"name": "Ravi", "roll": 2, "house": "Blue"},
    {"name": "Zoya", "roll": 3, "house": "Green"}
]

print(students[1]["house"])
```

- ( ) Red
- ( ) Green
- (•) Blue
- ( ) Ravi

6) What type of data structure is students in the code used by Prof. Andrew? ***1 point***

- ( ) Dictionary of Lists
- ( ) Tuple of Dictionaries
- (•) List of Dictionaries
- ( ) Dictionary of Tuples

7) What will happen if Prof. Andrew writes print(students["Ravi"]["house"])? ***1 point***

- ( ) It will print "Blue"
- ( ) It will cause an IndexError
- ( ) It will cause a TypeError
- (•) It will cause a KeyError

Assessment submitted.
X

Week 6 Programming Assignment : 2 (/noc25_cs103/progassignment?name=582)

Week 6 : Programming Assignment 3 (/noc25_cs103/progassignment?name=583)

Week 7 ()

Text Transcripts ()

Download Videos ()

Problem Solving Session - July 2025 ()

8) Prof. Andrew updates the students list by adding a new student record like this: ***1 point***

```
students.append({"name": "Meera", "roll": 4, "house": "Red"})
students[0]["house"] = "Yellow"
```

Now, what will be the output of the following code?

```
count = 0
for student in students:
    if student["house"] == "Red":
        count += 1
print(count)
```

- 2
- 1
- 3
- 0

9) Prof. Andrew wants to print the names of all students who belong to the **"Red" house**. Which of the following statements about how he should approach this task is correct? ***1 point***

- Loop over the marks dictionary and filter by house
- Loop over the students list and check the "house" key in each dictionary
- Use the key "Red" directly on the students list
- Use the values() method on the list to get all houses and then print names

10) If Prof. Andrew wants to find out **how many students scored above 90 in the first subject,** what is the correct approach? ***1 point***

- Check if marks[name][1] > 90 for all students
- Loop through the students list and check "marks" key in each dictionary
- Loop through the marks dictionary and check if the **first element** of the list is greater than 90
- Sort the marks dictionary by name before checking scores

Bubble Sort is a simple sorting algorithm that works by **repeatedly comparing** adjacent elements in a list and **swapping them** if they are in the wrong order.

Imagine arranging numbers from smallest to largest. In each pass through the list, the largest number **“bubbles up”** to its correct position at the end. This process continues until the entire list is sorted.

```
numbers = [5, 1, 4, 2, 8]
```

Assessment submitted.
X

- Compare 5 and 1 → swap
- Compare 5 and 4 → swap
- Compare 5 and 2 → swap
- Compare 5 and 8 → no swap

After the first pass, 8 has reached its correct place. More passes sort the rest.

Why is it called Bubble Sort?
Because with each pass, the **largest unsorted element "bubbles" to the top,** like a bubble rising in water.

11) What will be the output after the **first complete pass** of bubble sort on the following list? ***1 point***

```
arr = [9, 4, 7, 3, 2]
```

- (●) [4, 7, 3, 2, 9]
- ( ) [9, 7, 4, 3, 2]
- ( ) [4, 3, 2, 7, 9]
- ( ) [9, 4, 7, 3, 2]

12) How many total **swaps** will happen to fully sort the list below using bubble sort? ***1 point***

```
arr = [3, 2, 1]
```

- ( ) 1
- ( ) 2
- (●) 3
- ( ) 0

13) Consider this bubble sort implementation: ***1 point***

```
arr = [5, 1, 4, 2, 8]
for i in range(len(arr)):
    for j in range(len(arr)-1):
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]
print(arr)
```

What is the **output?**

- (●) [1, 2, 4, 5, 8]
- ( ) [1, 4, 2, 5, 8]
- ( ) [1, 2, 4, 8, 5]
- ( ) Infinite loop

Assessment submitted.
X

14) What is the minimum number of **passes** required to sort this list using bubble sort? ***1 point***

```
arr = [1, 2, 3, 4, 5]
```

- (•) 0
- ( ) 1
- ( ) 4
- ( ) 5

15) In which of the following cases will **bubble sort perform the maximum number of *1 point* comparisons?**

- ( ) Already sorted list
- (•) List sorted in reverse order
- ( ) List with all elements the same
- ( ) List with one element

16) What will be the output of the following code? ***1 point***

```
arr = [2, 3, 1, 5, 4]
for i in range(len(arr)-1):
    for j in range(len(arr)-1-i):
        if arr[j] < arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]
print(arr)
```

- ( ) [5, 4, 3, 2, 1]
- ( ) [4, 5, 3, 2, 1]
- ( ) [5, 3, 4, 2, 1]
- (•) [2, 3, 1, 5, 4]

17) Which of the following would improve bubble sort's performance on already sorted lists? ***1 point***

- ( ) Increasing the number of passes
- (•) Using a flag to break early if no swaps occur
- ( ) Always sorting in reverse
- ( ) Swapping even if elements are equal

18) You have a list: ***1 point***

arr = [10, 20, 30, 40, 25]

After how many **passes** will 40 reach its correct position using bubble sort?

- (•) 1

Assessment submitted.
X

- 2
- 3
- 4

19) What will be the final output of the following variant of bubble sort? ***1 point***

```
arr = [6, 3, 5, 2, 1]
n = len(arr)
for i in range(n):
    for j in range(i+1, n):
        if arr[i] > arr[j]:
            arr[i], arr[j] = arr[j], arr[i]
print(arr)
```

- [1, 2, 3, 5, 6]
- [2, 1, 3, 5, 6]
- [6, 5, 3, 2, 1]
- [1, 3, 2, 5, 6]

20) Prof. Andrew wanted to make his Bubble Sort more efficient by adding a swapped flag to **break early** if no swaps happen during a pass. ***1 point***

Here's the code:

```
arr = [1, 2, 3, 4, 5]
n = len(arr)

for i in range(n):
    swapped = False
    for j in range(n - 1 - i):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
            swapped = True
    if not swapped:
        break
print(i)
```

What will be the output?

- 0
- 1
- 4
- 5

It was the day before the annual sports day at college. Prof. Andrew was helping students line up for the march past, but he had a problem. The students were standing in random order based on their heights, and he wanted them arranged from **shortest to tallest.**

Assessment submitted.
X

Instead of asking them all to sort themselves, Prof. Andrew used a simple strategy — something computers also use. It's called **Selection Sort.**

What is Selection Sort?

Selection Sort is a sorting algorithm where we select the smallest (or largest) item from the unsorted part of the list and put it at the correct position in the sorted part — step by step.

It's like arranging students in height order one at a time.

Let's say these are the student heights:

heights = [165, 150, 172, 160]

Step 1:

- Look through the entire list to find the **shortest student** (150).
- Swap it with the **first** student in line.
- Now the line looks like: [150, 165, 172, 160]

Step 2:

- From the remaining (unsorted) list [165, 172, 160], find the shortest → 160.
- Swap it with the second position.
- Now: [150, 160, 172, 165]

Step 3:

- From [172, 165], find the smallest → 165.
- Swap with third position → [150, 160, 165, 172]

Now the list is fully sorted!

21) Which of the following best describes the core idea of selection sort? ***1 point***

- ○ Swap every adjacent pair that is out of order.
- ○ Select the largest element and place it at the beginning.
- ◉ Select the smallest element and place it at the correct position in each pass.
- ○ Randomly shuffle until the list is sorted.

22) How many swaps are made in total when sorting [4, 3, 2, 1] using selection sort? ***1 point***

- ○ 6
- ○ 3
- ○ 4
- ◉ 2

23) What will be the output of the following code? ***1 point***

Assessment submitted.
X

```
arr = [5, 1, 4, 2]
for i in range(len(arr)):
    min_idx = i
    for j in range(i+1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
print(arr)
```

- (●) [1, 2, 4, 5]
- ( ) [5, 4, 2, 1]
- ( ) [1, 4, 2, 5]
- ( ) [1, 2, 5, 4]

24) Which one of the following is **true** about the number of comparisons in selection sort? ***1 point***

- ( ) It depends on whether the list is sorted or not
- ( ) It varies based on data size and value
- (●) It always performs the same number of comparisons
- ( ) It stops early if no swap is needed

25) What happens if all elements in the list are already equal? ***1 point***

- ( ) Selection sort performs no swaps or comparisons
- ( ) Selection sort skips sorting
- (●) It performs comparisons but makes zero swaps
- ( ) It performs swaps with random elements

26) What is the **output after the second pass** of selection sort on this list? ***1 point***

arr = [9, 3, 7, 1, 5]

- ( ) [3, 1, 7, 9, 5]
- ( ) [1, 3, 7, 5, 9]
- (●) [1, 3, 7, 9, 5]
- ( ) [1, 3, 5, 7, 9]

27) ***1 point***

```
arr = [5, 1, 4, 2]
for i in range(len(arr)):
    min_idx = i
    for j in range(i+1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
print(arr)
```

Assessment submitted.
X

What is wrong with this code?

- ○ Nothing, it works fine
- ○ It doesn’t place the smallest element correctly
- ○ It overwrites values and loses elements
- ◉ It misses the last element in sorting

28)

```
arr = [6, 3, 8, 5]
for i in range(len(arr) - 1):
    min_idx = i
    for j in range(i + 1, len(arr) - 1):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
print(arr)
```

1 point

What's the problem here?

- ◉ Inner loop misses comparing the last element
- ○ It swaps elements incorrectly
- ○ Outer loop runs too many times
- ○ List becomes reversed

29)

```
arr = [9, 7, 5, 3]
for i in range(len(arr)):
    min_idx = 0
    for j in range(i + 1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
print(arr)
```

1 point

What is the mistake here?

- ○ The code doesn't swap anything
- ◉ The wrong element is selected as the minimum
- ○ min_idx is not reset correctly
- ○ The loop condition is wrong

Assessment submitted.
X

30)

```
arr = [1, 2, 3, 4]
for i in range(len(arr)):
    min_idx = i
    for j in range(i + 1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
print(arr)
```

1 point

This code runs correctly, but what small optimization could improve performance?

- ○ Remove the outer loop
- ◉ Break if the list is sorted
- ○ Check if min_idx != i before swapping
- ○ Change selection sort to bubble sort

You may submit any number of times before the due date. The final submission will be considered for grading.

Submit Answers